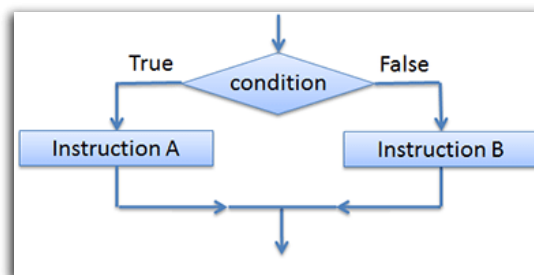


TP4 Cellular Automata

Hervé Wozniak - Université de Montpellier

Academic year 2024-2025



M1 Physics
HAP708P

Hervé Wozniak - Université de Montpellier

Table of contents

Objectives	3
Introduction	4
I - Basics of cellular automata	5
1. Wolfram notation	5
II - Implementation	7
1. Exercise 1	7
2. Exercise 2	7
3. Exercise 3	7
4. Exercise 4	8
5. Exercise 5	8
Conclusion	10

Objectives

Creating and exploring cellular automata in Python

Introduction

Cellular automata (CA) are systems that evolve in space and time with the simplest possible rules of evolution. While their microscopic behaviour is usually simplistic (whether deterministic or probabilistic), the macroscopic behaviour has characteristics that resemble physical systems. This behaviour can be non-trivially complex, as in the case of simple thermodynamic systems (e.g. +1 or -1 spins) which, at the macroscopic level, reveal measurable properties that are not the simple sum of the microscopic properties (ferromagnetism, Curie temperature...).

CAs are therefore used as an alternative to physical models, essentially to represent dynamical systems. As an illustration, we can mention several physical problems that have been studied from the perspective of CAs:

- avalanches, snow transport by wind, movement of sand dunes;
- wave propagation;
- scattering, percolation;
- ferromagnetism;
- gas on a network (hydrodynamics);
- etc.

There are also applications in biology, chemistry, sociology, environment, etc.

Some CAs have less simple rules in order to reproduce a given physical behaviour. But since the calculation of a CA is much faster than the integration of partial differential equations, it is possible to explore more finely the space of parameters and conditions that lead to certain behaviours.

Amongst CAs, those that are analogous to deterministic dynamical systems have been studied extensively. Indeed, they present the same sensitivity to initial conditions and for certain parameters, they can become chaotic, even producing fractal structures. It can be shown that some CAs exhibit strange attractors, structures discovered in Lorenz's (1963) model of atmospheric convection.

I Basics of cellular automata

In the one-dimensional (1D) case, a CA is made of a mesh (grid) of equidistant cells, numbered with index i , whose state is symbolized by s_i (unique property) which is 0 or 1.

This state s_i evolves as a function of (discrete) time t according to a rule that involves the state of neighbouring cells. For a CA of order k , the state s_i at time $t + 1$ depends on the state of the $k - 1$ nearest cells, as well as its own state, at the previous time t . In general k is odd to respect the symmetry of the neighbourhood. The simplest CAs uses $k = 3$ (i.e. one cell on the left and one on the right).

A CA is only complete when the **initial conditions** (values of the states of all the cells at time $t = 0$) and the **boundary conditions** (generally periodic) are also given.

Several generalizations are possible:

- the state s_i can take more than 2 values;
- the state change rule may not be unique (especially at the boundaries);
- the rule that allows the state of a cell to change according to the neighbourhood can be deterministic or stochastic (random);
- the change of state may be asynchronous (finite speed interaction);
- the range of the interaction (k) can increase but one can also consider a non-continuous neighbourhood (one skips every second cell for example...).

In the case of the Langton CA, there are 8 states and 29 rules; this CA reproduces the self-replicating operation of a living cell. And when we add a rule called "sterilization", we make corals by fossilization...

In this tutorial we will only see the simplest case (continuous neighbourhood, unique and deterministic rule, binary state, synchronous changes).

1. Wolfram notation

For a 1D CA of order $k = 3$, the state $s_i(t + 1)$ depends on the state of the triplet of cells: $s_{i-1}(t)$, $s_i(t)$ and $s_{i+1}(t)$. In other words: $s_i(t + 1) = \Phi(s_{i-1}(t), s_i(t), s_{i+1}(t))$.

This triplet of cells can take 8 values to which we associate $\alpha_k = 0$ or 1 according to the following list:

$$\underbrace{111}_{\alpha_7} \underbrace{110}_{\alpha_6} \underbrace{101}_{\alpha_5} \underbrace{100}_{\alpha_4} \underbrace{011}_{\alpha_3} \underbrace{010}_{\alpha_2} \underbrace{001}_{\alpha_1} \underbrace{000}_{\alpha_0}$$

Thus, the binary number formed by the values of the α_k (the least significant bit α_0 being on the right) represents the rule of change of states according to the neighbourhood. This 8-bit binary number $\alpha_7\alpha_6\alpha_5\alpha_4\alpha_3\alpha_2\alpha_1\alpha_0$ represents an integer between 0 and 255, called the CA **$\mathcal{R}$ rule**.

These 256 rules were studied by Wolfram (who introduced this notation) in 1983 and many others since.

Example :

To study a CA of rule 110 (thus 01101110 in binary) means that the rule of change of state is the following:

$$\underbrace{111}_0 \underbrace{110}_1 \underbrace{101}_1 \underbrace{100}_0 \underbrace{011}_1 \underbrace{010}_1 \underbrace{001}_1 \underbrace{000}_0$$

The rule often distinguishes the left side $i - 1$ from the right $i + 1$. But many rules are identical due to symmetries.

A CA is studied by varying time. A system that changes state as a function of time is a dynamical system. When space and time are continuous, statistical physics is used to predict the behaviour (Liouville equation, etc.).

For a CA, space and time are discrete. It is therefore a discrete dynamical system.

An important property of CAs is that the changes of state are synchronised (all states change at the same time).

💡 **Fundamental** :

From a programming point of view, a CA is a very simple object.

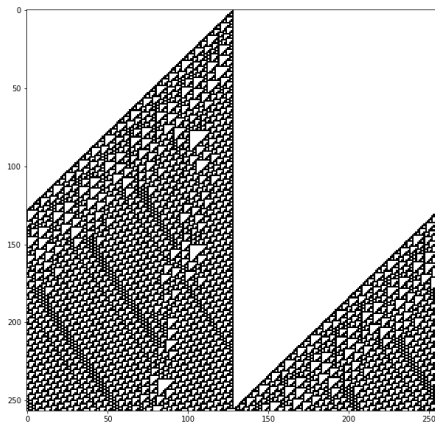
Time is discrete, so it is only a matter of iterating over a time variable. No differential equations.

Space is discrete, so no need to interpolate. No partial differential equations.

The states are 0 or 1, so the arithmetic is Boolean; there is **no loss of information** or **precision** in the calculations.

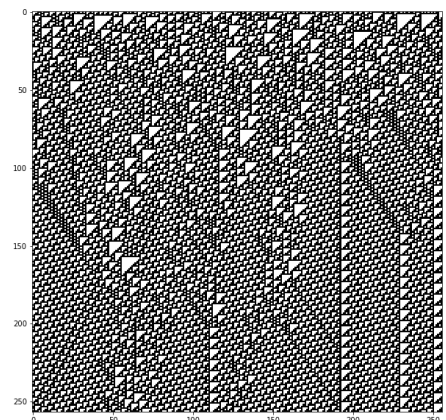
Example of rule 110

This example illustrates the importance of initial conditions.



```
ca(rule=110,    largeur=256,    niter=256,    order=3,
  init='centre')
```

```
ca(rule=110,    largeur=256,    niter=256,    order=3,
  init='random')
```



II Implementation

1. Exercise 1

Aim

Convert a rule $\mathcal{R}$ numbered according to the Wolfram convention into a Python `list` of 0s and 1s containing the 8-bit binary representation, with the least significant bit on the left so that it corresponds to the `[0]` element of the `list`.

As input, the function takes the name of the rule and the order of the CA ($k = 3$ in the Wolfram case, but the function is easily generalized to handle any value of k).

```
1 >>> rule_to_binlist(126,order=3)
2 [0, 1, 1, 1, 1, 1, 1, 0]
3 >>> rule_to_binlist(110,order=3)
4 [0, 1, 1, 1, 0, 1, 1, 0]
```

2. Exercise 2

Aim

This involves writing a function which, for a k -tuple of input cell states, returns the conversion to an integer value (`int`).

In other words, for an automaton of order 3, the triplet in the form of `list` `[0,1,1]` returns the value `int` 3 which will be interpreted as α_3 :

```
1 >>> state([0,1,1])
2 3
3 >>> state([1,0,1])
4 5
```

3. Exercise 3

Aim

Consider a list of states of length N which constitutes the initial condition of the CA.

A function (called *sliding window*) is applied to this list. It is called *window* because it returns an extraction of k values from the original list centred on a given cell. It is called *sliding* because each time it is called, the extraction is shifted one cell to the right (increasing `list` index).

Warning :

At iteration #0, this function returns a list of k values with cell 0 in the centre. This implies that $\frac{k-1}{2}$ values are to the left of 0.

To achieve this condition, *periodic boundary conditions* are applied. We will therefore copy the values which are on the far right of the list.

At iteration i , it returns a list of k values centred on cell i .

As the last iterations approach, the list is completed with the values on the left of the list.

⊕ Extra : Advanced Python

Instead of calling a function that returns only one value N times (a function ending in `return`), a generative function ending in `yield` can be used to advantage, which, combined with a loop in the function, allows it to be called only once.

Practice the example below before using it for CA.

```
1 def test_yield(niter):
2     for i in range(niter):
3         yield i
4
5 for j in test_yield(10):
6     print(j)
7
```

Applied to the requested function, this can result in the following behaviour, where we see that the returned list contains all possible triplet windows:

```
1 >>> print(list(moving_window([1,2,3,4,5,6], wsize=3)))
2 [[6, 1, 2], [1, 2, 3], [2, 3, 4], [3, 4, 5], [4, 5, 6], [5, 6, 1]]
```

4. Exercise 4

Aim

Since CA is defined as the application of a rule on a k -tuple, we now need to create a function that performs the state changes.

For an input k -tuple, the function returns the k -tuple modified by the rule. Since the cells of an CA change state synchronously, we can take advantage of this function to browse all the cells. Thus, we apply the function to the complete list of cells. The output is the updated list with the new states.

We will take advantage of the previous exercises:

- exercise 2 allows you to find the value of α which will replace the old state;
- exercise 3 moves a window of width k along the list of cells.

```
1 >>> print(apply_rule([0,0,1,0,0], rule_to_binlist(rule_name=110, order=3), order=3))
2 [0, 1, 1, 0, 0]
```

5. Exercise 5

Aim

All the basic functions have been constructed. We now need to plan the advancement in time, which takes place in the form of synchronized iteration for all the cells.

We therefore write a function which, at each call, advances the CA by one time step.

In other words, at a time $t + 1$, the function applies the rule $\mathcal{R}$ on the state of the cells taken at time t .

⚠ Warning :

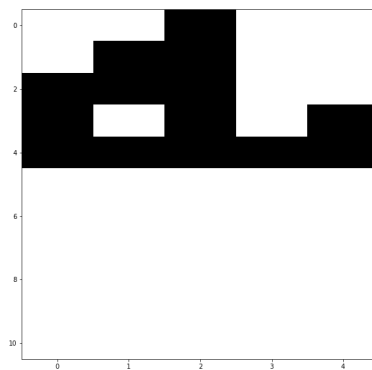
Moving from left to right, we must ensure that the state of the left cell(s) is taken at time t and not $t + 1$. A priori this problem should have been solved in exercise 4.

Example of an expected outcome:

```
1 >>> output=ca_iterate(init_cond=[0,0,1,0,0], rule=110, order=3, niter=10)
2 >>> print(list(output))
3 [[0, 0, 1, 0, 0], [0, 1, 1, 0, 0], [1, 1, 1, 0, 0], [1, 0, 1, 0, 1], [1, 1, 1, 1, 1], [0,
  0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0,
  0, 0, 0]]
```

The iteration function can return a list of lists (or a *generator* object produced by the `yield` statement) which is then plotted as follows:

```
1>>> import numpy as np
2>>> tab=np.array(list(output))
3>>> import matplotlib.pyplot as plt
4>>> plt.figure(figsize=(10,10))
5>>> plt.imshow(tab,origin="upper",cmap="Greys",aspect='auto')
```



Conclusion

We might see other types of CAs in the rest of the practical exercises (probabilistic CAs, 2D, totalizers, Q2R, forest fire propagation or epidemics, etc.). For some rules, the result of a CA depends strongly on the initial conditions. It's up to you.

Cosmetic rules :

- 150, 195, ... Sierpinski's triangles
- 184 : coverage of the TP
- 90 : converges to a stable state at length/2
- 110 : structures appear according to the initial conditions
- 17, 19, 23, etc... 17, 19, 23, etc. : with random initial conditions but to be compared with a 1 in the centre